

SHOP-BASED E-COMMERCE PLATFORM

Multi-Vendor Local Marketplace

A platform where local wholesalers and brand retailers each run their own independent virtual shop — for buyers who want a specific seller, not a random one.

Project Proposal Presentation

1. Patel Tarang G. 2. Gameti Jinil M. 3. Panchal Harsh K.
4. Patel Jenil K. 5. Patel Shubham H. 6. Patel Shivang C.

Vishwakarma Government Engineering College · Information Technology

INTRODUCTION

The Problem We're Solving

Most e-commerce platforms dump every seller's products into one flat, mixed catalogue. That breaks how local wholesale and brand retail actually works — a customer who wants a specific grocer's brand ends up scrolling past unrelated sellers' versions of the same item.

Who feels this? Local wholesalers, brand owners, and neighbourhood shop owners who want a real online storefront customers can find directly — without competing inside a mixed listing.



Typical Marketplace

One flat catalogue — every seller's products mixed together, buyers can't shop a specific store.



Our Idea: Shop-Based

Every approved seller runs an independent virtual shop. Buyers enter a shop and browse only its products — like visiting a physical store.

Goal: an online storefront model, not a mixed listing model.

PROJECT GOALS

Objectives

1

Independent Seller Shops

Let sellers register, get admin-approved, and run their own independent shop.

2

Isolated Catalogues

Keep each shop's product catalogue isolated and visible only within that shop.

3

Simple Buyer Journey

Let buyers discover shops, browse by shop, add to cart, and place orders.

4

Lean, Practical Stack

Keep the stack minimal: plain React (JavaScript), Node/Express, MongoDB only.

5

Image-Heavy Catalogues

Support product photos at scale without slowing the platform down.

UNDER THE HOOD

Technology Stack

Layer	Technology	Why It Fits
Frontend	React.js (JavaScript)	Fast dev, component-based UI, no TS overhead
Styling	Tailwind CSS	Utility-first, quick to build, no separate CSS files
State Mgmt	Context API + useReducer	Enough for cart/auth; skips Redux boilerplate
Backend	Node.js + Express.js	JS everywhere (MERN-style), simple REST layer
Database	MongoDB (only)	Schema-flexible — fits shops/products/orders well
Auth	JWT + bcrypt	Stateless auth, role-based access control
Images	Cloudinary CDN	Free tier, auto-optimized image delivery
Payments	Razorpay Payment Links	India-first, fast to integrate, no PCI burden
Deployment	Render / Vercel	Free-tier friendly, auto-deploy from Git

User Roles

Buyer

Browses the shop directory, opens a shop, views its products, adds to cart, places orders, tracks status.

Seller

Registers → waits for admin approval → sets up shop profile → uploads products → fulfils orders.

Admin

Approves/rejects seller applications, manages categories, moderates listings, views platform-wide orders.

PLANNED FEATURE SET

Buyer-Side Features

1

Browse the directory of all approved shops

2

Open a specific shop and view only that shop's products

3

Search and filter products within a shop

4

Add products to cart (per-shop or multi-shop split at checkout)

5

Place an order and track status: placed → confirmed → shipped → delivered

6

Buyer profile with full order history

PLANNED FEATURE SET

Seller-Side Features

1

Seller registration with shop details (name, brand, address)

2

Approval workflow — shop goes live only after admin approval

3

Seller dashboard: add / edit / delete products with images

4

Stock and price management per product

5

View and update incoming orders for their own shop

6

Public shop profile page shown to buyers

PLANNED FEATURE SET

Admin-Side & Platform-Wide Features

ADMIN

- Approve or reject seller/shop registration requests
- Manage product categories
- View all shops, all orders, and basic platform stats
- Suspend or remove a shop if needed

CROSS-CUTTING

- JWT-based authentication with role-based route protection
- Shop-scoped product isolation — enforced at DB + API level
- Image upload via Multer + Cloudinary for products & shop logos
- Cart and checkout flow with order creation

MONGODB SCHEMA

Database Design

Only MongoDB is used — no SQL database anywhere in the stack. Six collections, linked with ObjectId references.

users

name, email, password, role
(buyer / seller / admin)

shops

sellerId, shopName, shopLogo,
isApproved

products

shopId, name, brand, price,
stock, images[]

carts

buyerId, items[{productId,
shopId, qty}]

orders

buyerId, shopId, products[],
totalAmount, status

categories

name (managed by admin)

products.shopId is the key that enforces catalogue isolation — every product belongs to exactly one shop.

SYSTEM DESIGN

Architecture & Image Upload Flow

PROJECT STRUCTURE

client/ → React frontend (buyer, seller, admin pages)

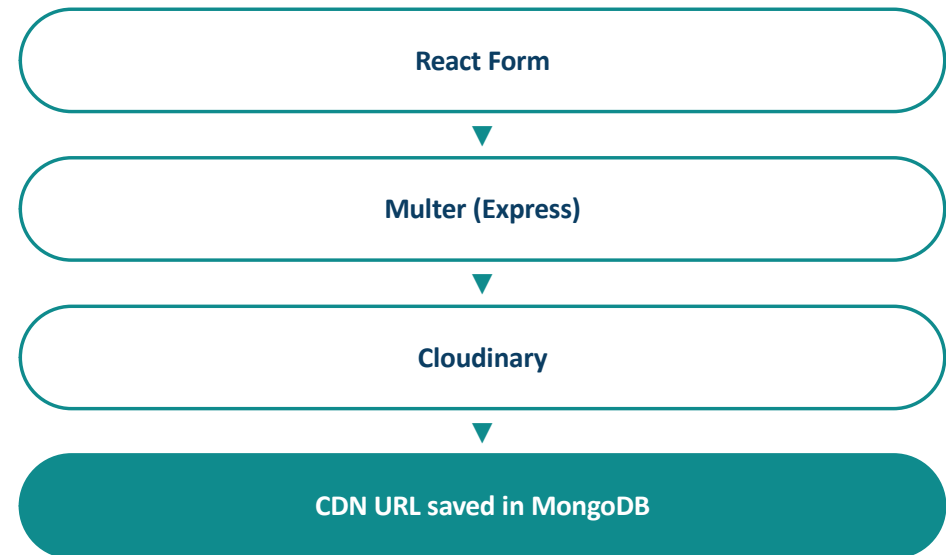
server/ → Express API (routes, controllers, models)

server/models/ → User, Shop, Product, Cart, Order, Category

server/middleware/ → auth, role-check, upload

docs/ → report, ER diagram, screenshots

IMAGE UPLOAD FLOW



Only the CDN URL is stored in MongoDB — never raw image files — keeping the DB light and image traffic off the app server.

Real-World Usability & Future Scope

WHO WILL USE THIS?

- Local wholesalers who want a direct storefront
- Brand owners selling their own product line
- Neighbourhood shops (grocery, clothing, footwear) going online
- Buyers who want a specific seller's brand, not a random match

FUTURE SCOPE (v2)

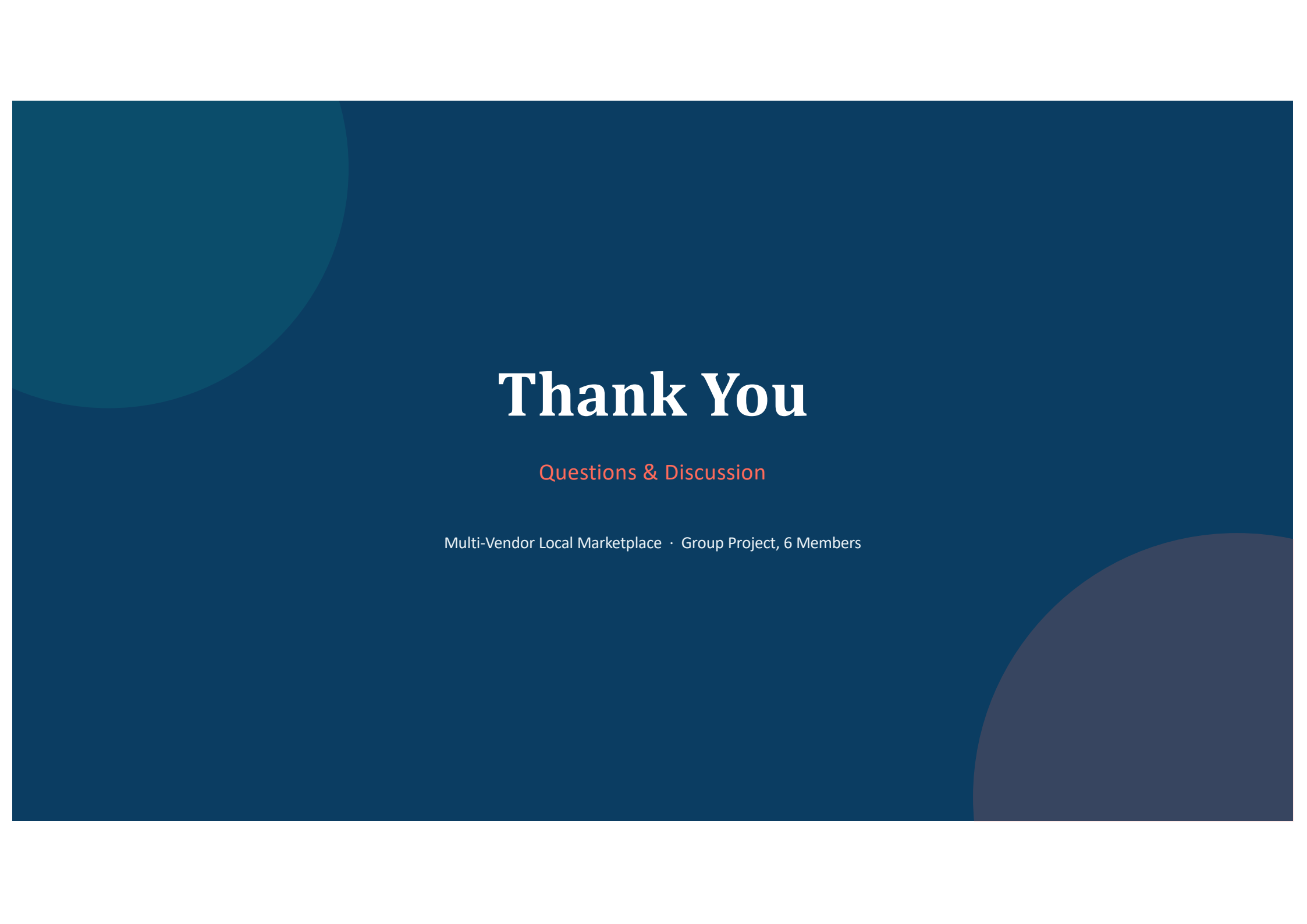
Ratings & reviews

Delivery tracking

Seller analytics dashboard

Wishlist & coupon system

Not yet built — this deck proposes the problem, the solution, and the exact tools we plan to use before development begins.



Thank You

Questions & Discussion

Multi-Vendor Local Marketplace · Group Project, 6 Members